Reviewer Pack — Schur-Weyl Multiplicity Gap in Permanent vs Determinant Tens…

Entry 4b47ab077abe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 04:36:58 UTC

Schur-Weyl Multiplicity Gap in Permanent vs Determinant Tensor Powers

Entry ID: `4b47ab077abe` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-01 04:36:58 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Permanent Polynomial

Statement:

> For all n ≥ 2, the multiplicity of the trivial representation in the decomposition of S^k(perm_n) exceeds that in S^k(det_m) by a factor of 2^{Ω(n)} for m = n^{1.5} and k = log n.

Rationale (proposer's reasoning):

> Schur-Weyl duality decomposes symmetric powers into irreducible representations, with trivial representation multiplicities reflecting symmetry-breaking complexity. A gap here could indicate structural separation between permanent and determinant orbit closures under linear substitutions.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `94a5538f865a8870`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=5.9s

5.1 Generated Python source

```python
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_mult(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def matrix_power(A, k):
    n = len(A)
    result = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
    while k:
        if k % 2 == 1:
            result = matrix_mult(result, A)
        A = matrix_mult(A, A)
        k //= 2
    return result

def gaussian_elimination(A):
    n = len(A)
    rank = 0
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
```

```
        if A[i][i] == 0:
            continue
        rank += 1
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return rank

def trivial_representation_multiplicity(A):
    return gaussian_elimination(A)

def generate_random_3cnf(n):
    clauses = []
    for _ in range(10*n):
        clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(3)]
        if len(set(clause)) == 3:
            clauses.append(clause)
    return clauses

def permanent_polynomial(clauses):
    n = max(abs(x) for x in sum(clauses, []))
    A = [[0] * (n+1) for _ in range(n+1)]
    for clause in clauses:
        for x in clause:
            if x > 0:
                A[x-1][x] += 1
            else:
                A[-x-1][-x] += 1
    return matrix_power(A, n)[0][n]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = int(n ** 1.5)
    k = math.ceil(math.log2(n))

    perm_n = generate_random_3cnf(n)
    det_m = generate_random_3cnf(m)

    perm_poly = permanent_polynomial(perm_n)
    det_poly = permanent_polynomial(det_m)

    trivial_perm = trivial_representation_multiplicity(perm_poly)
    trivial_det = trivial_representation_multiplicity(det_poly)

    ratio = trivial_perm / trivial_det

    return {
        "metric_name": "trivial_representation_ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": ratio >= 2 ** (n / 10),
        "counterexample": "" if ratio >= 2 ** (n / 10) else f"Ratio {ratio} < 2^{n/10}"
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30)) + [101, 103, 107]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r for r in results if not r["conjecture_holds"])["seed"]
        print(f"RESULT: FALSIFIED counterexample=\"Ratio < 2^{n/10}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff917c49.py", line 116, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff917c49.py", line 97, in run_trial
    trivial_perm = trivial_representation_multiplicity(perm_poly)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff917c49.py", line 64, in trivial_repre
    return gaussian_elimination(A)
           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff917c49.py", line 46, in gaussian_elim
    n = len(A)
        ^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in Gaussian elimination, preventing data collection | next: Fix the gaussian_elimination function to handle integer inputs properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	77348
2	preregistration	ollama_remote	qwen3:8b	0	0	24228
3	novelty	ollama_remote	qwen3:8b	0	0	20780
4	novelty	ollama_remote	qwen3:8b	0	0	16091
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15256
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11768
7	judge	ollama_remote	qwen3:8b	0	0	14024

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 179495 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/4b47ab077abe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4b47ab077abe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4b47ab077abe.tar.gz` (if generated)